

Deep Dive into Vector Databases and ANN Algorithms

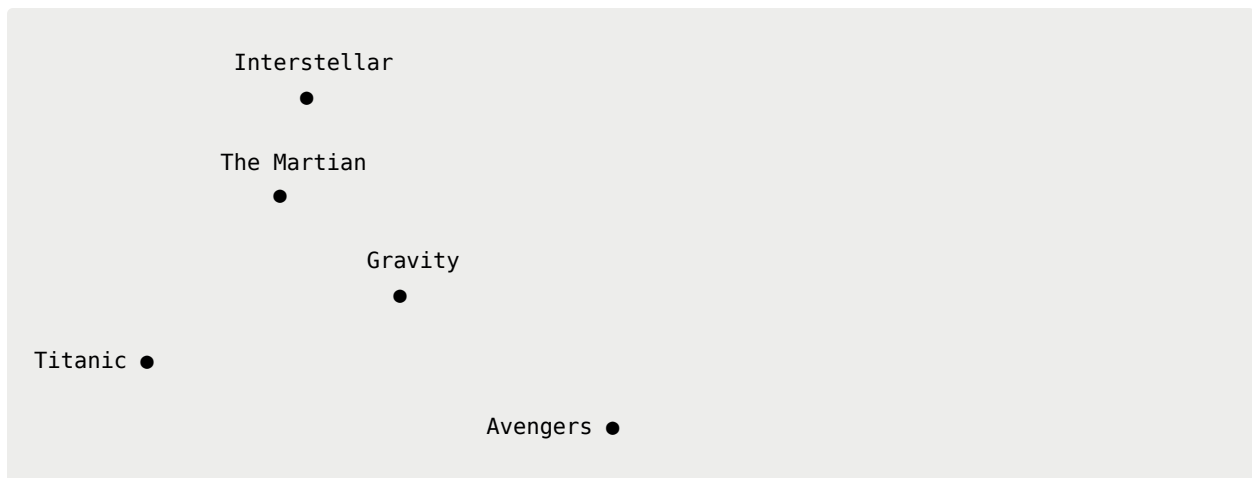
Deep Dive into Vector Databases and ANN Algorithms

Modern AI applications often need to search through millions—or even billions—of embeddings.

The basic property that makes vector search useful is:

Semantically similar information tends to produce vectors that are located near each other in vector space.

Conceptually:



Real embedding spaces may contain hundreds or thousands of dimensions, so we cannot visualize them directly.

But the basic idea remains the same:

Similar meaning
↓
Similar vectors

↓
Nearby locations in vector space

1. The Basic Vector Search Problem

Suppose our database contains:

10,000,000 movie embeddings

Each movie has a vector:

```
Movie A → [0.12, 0.87, -0.34, ...]  
Movie B → [0.78, 0.11, 0.49, ...]  
Movie C → ...  
...  
10 million movies
```

A user searches:

"movies about humans exploring deep space"

The embedding model converts the query into another vector:

```
Query Vector  
↓  
[0.19, 0.71, -0.29, ...]
```

Our task is:

Find the Top 10 nearest vectors.

The most obvious solution is:

Query
↓
Compare with Movie 1
Compare with Movie 2
Compare with Movie 3
...
Compare with Movie 10,000,000
↓
Keep the Top K
↓
Return nearest 10

This is called:

Exact Nearest Neighbor Search

Conceptually:

```
for every vector V:  
    similarity = cosine(query, V)  
  
return highest 10
```

The biggest advantage of exact search is simple:

It does not guess.

If the distance function and dataset are fixed, exact search finds the mathematically nearest items.

So there is nothing mathematically wrong with exact KNN.

The problem is **computational cost**.

2. Why Exact KNN Becomes Expensive

Suppose:

N = 10,000,000 vectors
D = 1,536 dimensions

For cosine similarity, we perform calculations across every dimension.

Very roughly:

$10,000,000 \times 1,536$

That means more than:

15 billion

dimension-level operations in the naïve picture.

And that is for **one query**.

With thousands of users sending queries continuously, brute-force search quickly becomes expensive.

3. Do We Really Need to Search Everything?

Imagine standing in Delhi and asking:

| Find me the nearest coffee shop.

You would not calculate the distance between yourself and every coffee shop in India.

You already know that:

Delhi → promising
Mumbai → probably irrelevant
Bangalore → probably irrelevant
Chennai → probably irrelevant

Large areas can be eliminated before detailed comparisons begin.

That same idea leads to:

ANN — Approximate Nearest Neighbor Search

Instead of asking:

| How do I calculate the distance to every vector faster?

ANN asks:

| How can I avoid considering most vectors in the first place?

4. Exact Search vs Approximate Search

Suppose exact search returns:

1. Interstellar
2. The Martian
3. Gravity

An approximate algorithm returns:

1. Interstellar
2. The Martian
3. Apollo 13

It missed:

Gravity

and returned:

Apollo 13

For something such as movie recommendations, this may still be perfectly useful.

The trade-off becomes:

Exact Search

High accuracy

but

high computational cost

VS.

Approximate Search

Very high accuracy

but

much lower computational cost

5. What ANN Actually Approximates

ANN does not necessarily calculate distances incorrectly.

Often, the approximation happens earlier.

Instead of examining:

10,000,000 candidates

an ANN index might identify only:

10,000 promising candidates

and then perform proper distance calculations on those candidates.

This distinction is important.

ANN may return the wrong neighbor because:

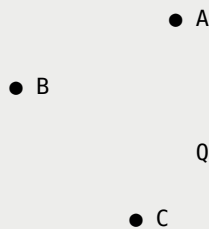
The correct vector was never examined.

not because:

Its distance was calculated incorrectly.

6. Why ANN Can Miss the True Nearest Neighbor

Imagine:



Suppose the ANN structure decides that the query belongs to a particular region and searches only:

A
C

But **B** happens to be just outside the selected region.

If **B** was actually slightly closer than **A**, ANN misses the correct answer because:

```
distance(Q, B)
```

was never calculated.

This is where the word **approximate** comes from.

7. Recall@K

Suppose exact search tells us that the true nearest 10 vectors are:

```
A B C D E F G H I J
```

ANN returns:

```
A B C D E F G H X Y
```

Eight correct neighbors were recovered.

Therefore:

```
Recall@10 = 8 / 10  
           = 0.8  
           = 80%
```

In general:

```
           True nearest neighbors recovered  
Recall@K = -----  
                K
```

Recall is commonly used when evaluating ANN systems.

8. ANN Is Not Simply “Inaccurate”

Think of ANN as a spectrum.

Exact search:

```
Search almost everything  
    ↓  
Very high recall
```

↓
More work

ANN configuration A:

Search a very small region
↓
Lower recall
↓
Very fast

ANN configuration B:

Explore more candidates
↓
Higher recall
↓
Slightly more work

Most ANN algorithms therefore expose some way of controlling:

How much of the search space should I explore?

This gives us the central ANN trade-off:

Speed ↔ Recall

9. Two Major ANN Ideas

Different ANN algorithms solve the candidate-selection problem differently.

Two important approaches are:

IVF
↓
Divide vector space into regions

and:

```
HNSW
↓
Navigate through a graph of vectors
```

Part 1 — IVF

10. IVF Intuition: Divide the Vector Space

Suppose embeddings naturally form regions:

```
Interstellar •
              • The Martian
              • Gravity

Titanic •
        • The Notebook

              Avengers •
              • Iron Man
              • Captain America
```

Conceptually:

```
Space movies    → one region
Romance movies  → another region
Superhero movies → another region
```

Real vectors may have:

```
1,536 dimensions
3,072 dimensions
...
```

but the same geometric intuition applies.

Instead of treating the database as one giant list, we divide the vector space into smaller regions.

11. Dividing One Million Vectors

Suppose we have:

```
1,000,000 vectors
```

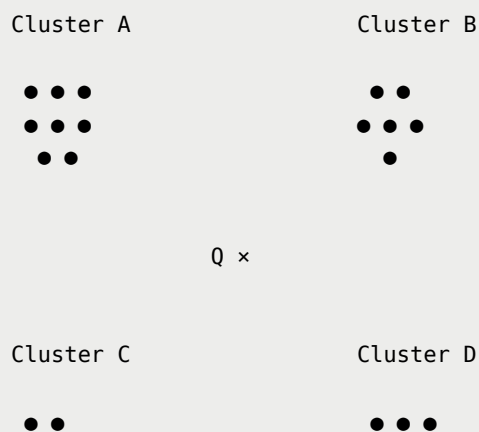
We divide them into:

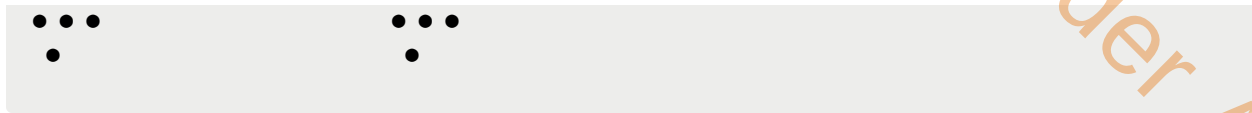
```
Cluster 1  
Cluster 2  
Cluster 3  
...  
Cluster 1,000
```

If they were perfectly balanced for this simplified example:

```
~1,000 vectors per cluster
```

Conceptually:





When query **Q** arrives, we do not immediately compare it against all one million vectors.

Instead:

```
Which cluster appears closest to Q?
```

Suppose the answer is:

```
Cluster D
```

Then we search inside Cluster D.

Instead of:

```
1,000,000 vector comparisons
```

we move toward:

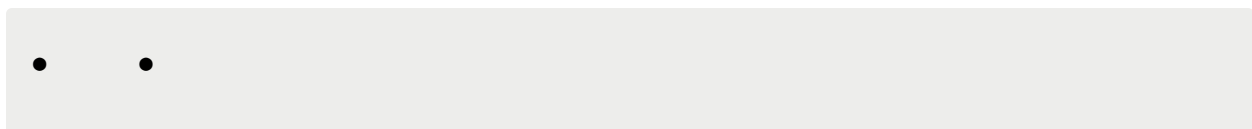
```
Find promising cluster  
+  
Search ~1,000 vectors
```

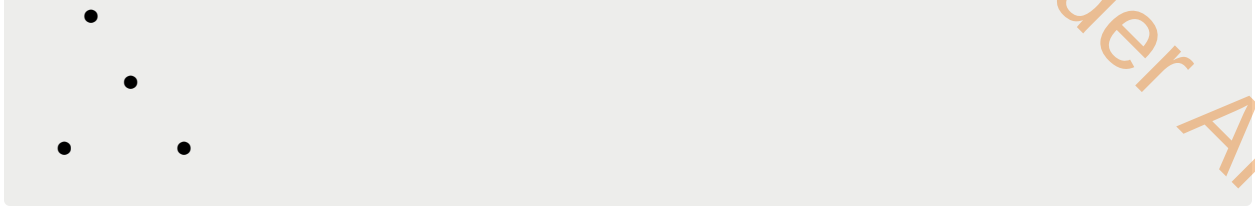
This is the basic intuition behind IVF.

12. Centroids

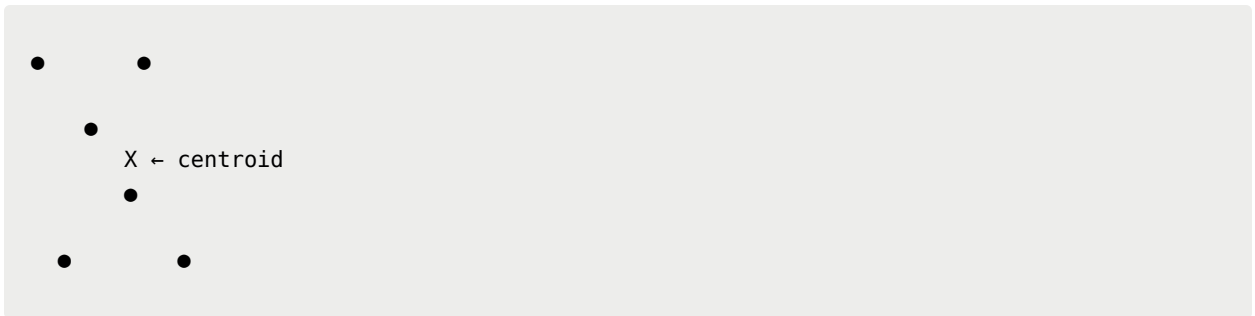
How do we represent a cluster?

Consider:





We calculate a representative central position:



The centroid can be thought of as:

| The rough address of a neighborhood.

For example:

```
Cluster A → Centroid CA
Cluster B → Centroid CB
Cluster C → Centroid CC
Cluster D → Centroid CD
```

Now:

```
1,000,000 vectors
    ↓
1,000 regions
    ↓
1,000 centroids
```

At query time, we initially compare the query against the centroids rather than all one million vectors.

13. IVF Index Construction

The clustering work does not happen from scratch for every query.
It happens while building the index.

Conceptually:

```
INDEX BUILD

1,000,000 vectors
  ↓
Group vectors into regions
  ↓
Calculate centroids
  ↓
Assign every vector to a region
  ↓
Store the groups
```

The resulting index can then be reused for many searches.

14. Miniature IVF Example

Suppose our movie embeddings are represented using two dimensions.

Interstellar	[8,8]
Gravity	[8,7]
The Martian	[7,8]
Titanic	[2,2]
The Notebook	[2,3]
La La Land	[3,2]
Avengers	[8,2]
Iron Man	[7,2]
Thor	[8,3]

The index might organize them into:

```
List 1
-----
Titanic
The Notebook
La La Land
```

```
List 2
-----
Interstellar
Gravity
The Martian
```

```
List 3
-----
Avengers
Iron Man
Thor
```

Each region receives a centroid:

```
C1 → Romance region
C2 → Space region
C3 → Superhero region
```

15. Searching the IVF Index

Suppose the user searches:

```
"astronauts travelling through space"
```

The embedding model produces:

```
Q = [7.7, 7.5]
```

Instead of comparing Q with every movie, first calculate:

```
distance(Q, C1)
distance(Q, C2)
distance(Q, C3)
```

Suppose:

```
C1 = far
C2 = very close
C3 = moderately far
```

Therefore:

```
C2 looks promising
```

Search List 2:

```
Interstellar
Gravity
The Martian
```

Then calculate their actual similarities.

Example:

```
Interstellar → 0.94
The Martian  → 0.91
Gravity      → 0.86
```

Movies in the other regions were never evaluated.

That is where the performance improvement comes from.

16. Why Is It Called IVF?

IVF stands for:

Inverted File Index

The useful mental model is:

Centroid / Region
↓
List of vectors belonging to it

For example:

Centroid A
↓
[V3, V9, V81, V400, ...]

Centroid B
↓
[V2, V7, V33, V101, ...]

Centroid C
↓
[V1, V4, V18, V900, ...]

The search becomes:

Query
↓
Find promising centroid
↓
Get corresponding vector list
↓
Search that list

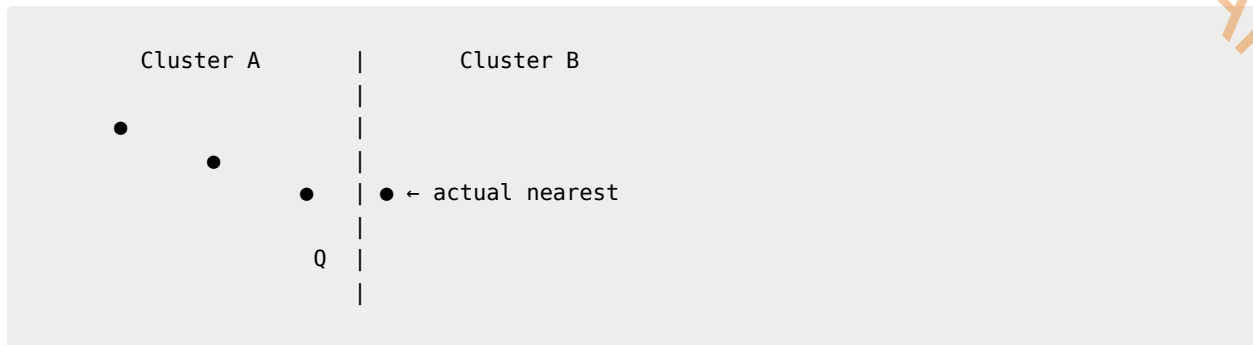
So the IVF mental model is:

Divide
↓
Locate promising regions
↓
Search those regions

17. The Boundary Problem

Searching only one cluster can create problems.

Imagine:



The query is slightly on the Cluster A side.

So the index searches:

Cluster A

But the actual nearest vector may be just across the boundary inside:

Cluster B

If Cluster B is never searched, that neighbor is missed.

Vector space does not contain real walls.

The clusters are an optimization structure imposed by the index.

18. Search Multiple Clusters

Instead of searching:

1 closest cluster

we can search:

5 closest clusters

or:

20 closest clusters

The relationship becomes:

```
More clusters searched
    ↓
More candidate vectors
    ↓
Lower chance of missing neighbors
    ↓
Higher recall
    ↓
More computational work
```

In IVFFlat, the number of lists searched can be controlled using the idea of **probes**.

Conceptually:

```
Few probes
    ↓
Faster search
    ↓
Potentially lower recall
```

versus:

```
More probes
    ↓
More work
    ↓
Potentially higher recall
```

19. IVF at Larger Scale

Suppose:

```
1,000,000 vectors
```

divided into:

```
1,000 lists
```

Assume approximately:

```
1,000 vectors per list
```

for a simplified balanced example.

Exact search:

```
~1,000,000 vectors inspected
```

IVF searching five promising lists:

```
5 × ~1,000
```

```
≈ 5,000 candidate vectors
```

plus the smaller centroid-selection work.

This is the fundamental advantage of region-based ANN search.

Part 2 — HNSW

20. HNSW: Hierarchical Navigable Small World

IVF divides vector space into clusters.

HNSW approaches the problem differently.

Instead of:

Divide the space into regions

HNSW asks:

| What if nearby vectors were connected to each other?

Search could then become **navigation**.

21. Start with a Graph

Suppose we have vectors:

A B
C D
E F

Connect nearby vectors:

A — B
| / |
C — D — E
 \
 F

Each vector is a:

Node

Connections between vectors are:

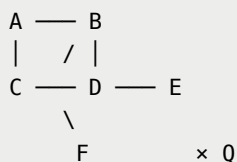
Edges

Conceptually:

A knows B and C
 B knows A, C and D
 D knows B, C, E and F
 ...

22. Search by Navigating the Graph

Suppose query Q is near F.



Instead of comparing Q against every vector, start somewhere in the graph.

Suppose we start at:

A

Calculate:

`distance(Q, A)`

A's neighbors are:

B
C

Suppose B is closer.

Move:

$A \rightarrow B$

Now inspect B's neighbors.

Suppose D is closer:

$A \rightarrow B \rightarrow D$

From D, suppose F is promising:

$A \rightarrow B \rightarrow D \rightarrow F$

We have navigated toward the area containing the query.

The basic idea is:

```
Start somewhere
  ↓
Inspect neighbors
  ↓
Move toward promising nodes
  ↓
Repeat
```

So:

Search becomes navigation.

23. The Problem with a Flat Graph

Suppose vectors form a very large graph:

```
A-B-C-D-E-F-G-H-I-J-K-L-M-N-O-P-Q-R-S-T...
```

If you start near A and the query is near Z, local connections could require many hops:

```
A → B → C → D → E → F → ...
```

HNSW solves this using hierarchy.

24. Road Analogy

When travelling a long distance, we do not use tiny local streets for the entire journey.

We use:

```
Local street
  ↓
Main road
  ↓
Highway
  ↓
Main road
  ↓
Local street
```

Why?

```
Highways
  ↓
Large jumps
```

while:

Local roads
↓
Precision

This is the intuition behind the hierarchical part of HNSW.

25. Multiple Graph Layers

Instead of one graph, HNSW creates multiple layers.

Conceptually:

Layer 3
A ----- M ----- Z

Very few nodes and large jumps.

Layer 2
A ----- F ----- M ----- S ----- Z

More nodes.

Layer 1
A -- C -- F -- H -- K -- M -- P -- S -- V -- Z

More detail.

Layer 0
A-B-C-D-E-F-G-H-I-J-K-L-M-N-O-P-Q-R-S-T-U-V-W-X-Y-Z

The bottom layer contains all vectors.

The overall pattern is:

```
Higher layers
↓
Fewer nodes
↓
Longer jumps
```

and:

```
Lower layers
↓
More nodes
↓
Fine-grained navigation
```

26. Querying an HNSW Index

Suppose the query is near:

```
x
```

Start from an entry node at the highest layer.

For example:

```
Layer 3
A ----- M ----- Z
```

Starting from A, M may move us much closer to the query.

```
A → M
```

Descend one layer:

Layer 2

A --- F --- M --- S --- Z

From M, perhaps S is closer:

M → S

Descend again:

Layer 1

... S -- V -- Z

Perhaps V is closer:

S → V

Finally, at the bottom layer:

... V-W-X-Y-Z

perform a more local search:

V → W → X

Overall:

```
Start high
↓
Make large jumps
↓
Reach approximately the correct area
↓
Move down a layer
```

↓
Make smaller jumps
↓
Move down again
↓
Perform fine-grained search

This is the core intuition behind HNSW.

27. What Does an HNSW Node Store?

Suppose:

```
Interstellar = V1
```

Its node may contain references to nearby vectors:

```
Interstellar
├─ The Martian
├─ Gravity
├─ Ad Astra
└─ Arrival
```

The connections can differ across different layers.

HNSW therefore stores more than just raw vectors.

It also stores graph relationships between those vectors.

28. HNSW Does Not Understand Meaning

This distinction is important.

The embedding model creates semantic geometry:

```
Embedding Model
↓
```

"Interstellar" and "The Martian"
receive nearby vector representations

HNSW only sees:

Vector A
Vector B
distance(A, B)

So:

Embedding model
↓
Creates semantic geometry

while:

HNSW
↓
Navigates that geometry efficiently

HNSW itself does not understand movies, language, meaning, or concepts.

29. Why Not Connect Every Node to Every Other Node?

Suppose:

1,000,000 vectors

If every node connected to every other node, each node could require roughly:

~1,000,000 connections

That would recreate a scalability problem.

We need:

```

Enough connections
  ↓
Good navigation
  
```

but:

```

Not too many connections
  ↓
Reasonable memory and computation
  
```

So HNSW creates a **sparse graph** with strategically selected neighbors.

30. How Vectors Appear Across Layers

Not every vector appears in every layer.

Conceptually:

```

Layer 3 → very few vectors
Layer 2 → more vectors
Layer 1 → many vectors
Layer 0 → every vector
  
```

For example:

Layer 3:	A		M	
Layer 2:	A	F	M	Z
Layer 1:	A B C	F G H	M N	... Z
Layer 0:	every vector			

Higher-layer membership is typically probabilistic.

The important intuition is:

Every vector exists at the base layer, while progressively fewer vectors appear in higher layers.

That creates the hierarchy.

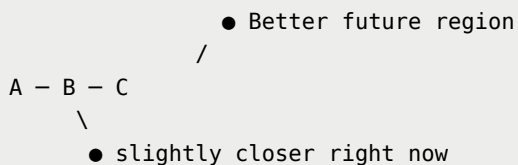
31. HNSW Is More Than Pure Greedy Search

A very simple greedy search would keep only:

one current node

and always move toward the single neighbor that looks best immediately.

That can create problems.



Following only the immediately best edge could trap the search in a local region.

Real HNSW search can maintain several promising candidates during exploration.

The useful intuition is:

HNSW does not blindly follow only one edge. It can explore multiple promising candidates to reduce the chance of becoming trapped in a poor local region.

32. HNSW Index-Build Cost

HNSW also requires work while constructing the index.

When a new vector  arrives:

Find promising neighbors
↓
Choose graph connections
↓
Insert X

If more work is spent finding good connections:

Better graph quality

but:

More expensive index construction

So HNSW has trade-offs during both:

BUILD TIME

and:

QUERY TIME

33. Number of Connections

Suppose each node connects to only:

2 neighbors

Advantages:

Lower memory usage

Fewer stored edges

Potential disadvantage:

Harder graph navigation

Now imagine:

100 neighbors per node

Navigation may become easier, but we also get:

More edges
More memory
More construction work
More neighbors to inspect

Many HNSW implementations use a parameter called:

M

A useful mental model is:

M $\approx$ how many graph connections
each node is allowed to maintain

34. Why HNSW Can Be Memory Hungry

IVF mainly stores grouping/index information.

HNSW stores:

Vectors
+

Graph edges
+
Multiple graph layers

With:

10 million vectors

and many neighbor references for each vector, those graph edges can consume significant memory.

HNSW can therefore achieve excellent query performance partly by spending more memory on its graph structure.

IVF vs HNSW

The main intuition can be summarized as:

IVF
|
├ Divide vector space into regions
├ Represent regions using centroids
├ Locate promising regions
└ Search vectors inside those regions

while:

HNSW
|
├ Represent vectors as graph nodes
├ Connect useful nearby nodes
├ Build multiple hierarchical layers
└ Navigate from broad search to local search

Both avoid checking every vector.

They simply avoid it in different ways.

Part 3 — Vector Compression

35. A Different Vector Problem: Storage

ANN helps solve:

| How do we search huge collections of vectors quickly?

But there is another problem:

| How do we store huge collections of vectors efficiently?

Suppose we have:

```
100,000,000 vectors
```

and each embedding contains:

```
1,536 dimensions
```

Assume each value is stored as:

```
float32
```

One float32 requires:

```
32 bits  
= 4 bytes
```

One vector therefore requires:

```
1536 × 4  
= 6144 bytes  
≈ 6 KB
```

Coder Army

For 100 million vectors:

```
6144 × 100,000,000  
≈ 614 GB
```

That is only for the raw vectors.

It does not include:

```
Metadata  
IDs  
Indexes  
HNSW graph edges  
Database overhead  
Replication
```

So vectors can be expensive both to **search** and to **store**.

36. Do We Need Full Precision?

An embedding might contain:

```
[  
  0.183742,  
 -0.729182,  
  0.029117,  
  0.441827,  
  ...  
]
```

with perhaps:

```
1,536 values
```

Each value is stored with relatively high precision.

But for nearest-neighbor search, do we always need that much numerical precision?

This leads to:

Quantization

The core idea is:

Represent high-precision values using lower-precision representations.

37. Simple Quantization

Suppose a 1,536-dimensional embedding uses float32.

Storage:

```
1536 × 4  
= 6144 bytes
```

Using float16:

```
1536 × 2  
= 3072 bytes
```

So storage becomes approximately:

```
2× smaller
```

Using something like one byte per value:

```
1536 bytes
```

which is approximately:

```
4× smaller
```

This is the general intuition behind quantization.
But Product Quantization takes the idea further.

38. Product Quantization — PQ

Take a tiny vector:

```
V = [  
  0.2,  
  0.8,  
 -0.4,  
  0.1,  
  0.7,  
 -0.2,  
  0.9,  
  0.3  
]
```

This vector has:

```
8 dimensions
```

Stored as float32:

```
8 × 4  
= 32 bytes
```

Product Quantization says:

| Instead of storing the entire vector precisely, divide it into smaller chunks.

39. Split the Vector into Sub-Vectors

Original:

```
[0.2, 0.8, -0.4, 0.1, 0.7, -0.2, 0.9, 0.3]
```

Split into four pieces:

```
Part 1  
[0.2, 0.8]  
  
Part 2  
[-0.4, 0.1]  
  
Part 3  
[0.7, -0.2]  
  
Part 4  
[0.9, 0.3]
```

These smaller pieces are called:

Sub-vectors

So:

```
8 dimensions  
  ↓  
4 sub-vectors  
  ↓  
2 dimensions each
```

40. Learn Common Patterns for Each Sub-Vector

Suppose we have one million complete vectors.

Take only the first sub-vector from each vector:

```
Vector A → [0.2, 0.8]  
Vector B → [0.3, 0.7]
```

```
Vector C → [-0.5, 0.2]
Vector D → [0.9, 0.1]
...
```

Many of these small vectors may look very similar.

For example:

```
[0.20, 0.80]
[0.22, 0.79]
[0.18, 0.84]
[0.21, 0.77]
```

Instead of storing every version independently, similar sub-vectors can be clustered.

41. Centroids and Codewords

Example:

```
Cluster 0

[0.20, 0.80]
[0.22, 0.79]
[0.18, 0.84]

Representative:

C0 = [0.20, 0.81]
```

Another cluster:

```
Cluster 1

[-0.42, 0.12]
[-0.39, 0.08]
[-0.45, 0.14]

Representative:
```

```
C1 = [-0.42, 0.11]
```

Each representative can be thought of as a:

Centroid

or:

Codeword

All the centroids together form a:

Codebook

42. The PQ Compression Trick

Instead of storing:

```
[0.22, 0.79]
```

we store something like:

Codeword 0

That means:

| Use centroid number 0 from this codebook.

Similarly, instead of:

```
[-0.39, 0.08]
```

we might store:

```
Codeword 1
```

The actual floating-point values are replaced with small integer identifiers.

43. Compressing a Complete Vector

Original:

```
[0.2, 0.8 | -0.4, 0.1 | 0.7, -0.2 | 0.9, 0.3]
```

Suppose PQ maps its four sub-vectors to:

```
Sub-vector 1 → centroid #17  
Sub-vector 2 → centroid #203  
Sub-vector 3 → centroid #81  
Sub-vector 4 → centroid #6
```

Instead of storing:

```
[0.2, 0.8, -0.4, 0.1, 0.7, -0.2, 0.9, 0.3]
```

we store:

```
[17, 203, 81, 6]
```

That is the compressed representation.

44. Why 256 Centroids Are Convenient

Suppose every subspace has:

256 possible centroids

Since:

$$256 = 2^8$$

we need:

8 bits

to identify one centroid.

And:

$$8 \text{ bits} = 1 \text{ byte}$$

Therefore each sub-vector can be represented using:

1 byte

45. PQ with a 1,536-Dimensional Embedding

Suppose:

$$D = 1536$$

Original float32 storage:

$$1536 \times 4$$

= 6144 bytes

For this example, split the vector into:

96 sub-vectors

Each sub-vector contains:

$1536 / 96$
= 16 dimensions

So:

1536 dimensions
↓
96 chunks
↓
16 dimensions per chunk

Suppose each chunk can be represented using one of:

256 centroids

Since selecting among 256 possibilities needs one byte:

1 chunk = 1 byte

We have:

96 chunks

Therefore the encoded vector requires:

96 × 1 byte
= 96 bytes

instead of:

6144 bytes

Compression ratio:

6144 / 96
= 64×

Conceptually:

Original

6144 bytes



PQ representation

96 bytes



This is why Product Quantization can dramatically reduce vector storage.

46. PQ Is Lossy Compression

Suppose the original sub-vector is:

[0.217, 0.794]

PQ might represent it using centroid:

[0.200, 0.810]

These are close, but they are not identical.

So PQ does not preserve the original vector perfectly.

It is:

Lossy Compression

The reconstructed value is an approximation.

Once again, vector systems use a trade-off:

```
Less storage
  ↑
Less numerical precision
```

Part 4 — What Makes a Vector Database?

A vector database is more than a collection of arrays.

A practical vector search system needs several layers.

47. Layer 1 — Original Data

For a movie:

```
ID: 101

Title:
Interstellar

Description:
A team travels through a wormhole...

Genre:
Sci-Fi
```

Year:
2014

This is ordinary application data.

48. Layer 2 — Vector Representation

The movie also has an embedding:

```
embedding:  
[0.21, -0.45, 0.81, ...]
```

Conceptually:

```
ID: 101  
Title: Interstellar  
Genre: Sci-Fi  
Year: 2014  
  
Vector:  
[0.21, -0.45, ...]
```

The vector allows semantic retrieval.

49. Layer 3 — Similarity or Distance Function

Given:

```
Query Vector = Q  
Movie Vector = M
```

we need to define:

How close are Q and M?

Possible choices include:

Cosine similarity
Dot product
Euclidean distance / L2

So a vector search system needs:

Vector storage
+
Similarity / distance function

50. Layer 4 — Vector Index

Without an index:

Query
↓
Compare against every vector

At large scale, this becomes expensive.

So we add an ANN index such as:

HNSW

or:

IVF / IVFFlat

Conceptually:

Vectors
↓
ANN Index
↓
Efficient candidate search

Coder Army

51. Layer 5 — Metadata

Applications almost never search using only semantic similarity.

Suppose the user asks:

| Recommend recent Hindi comedy movies similar to this movie.

We may need:

```
language = Hindi  
genre = Comedy  
year >= 2020
```

while also requesting:

```
nearest vectors to Query Q
```

Conceptually:

```
Find Top 10 vectors nearest to Q  
  
WHERE  
  
language = Hindi  
AND  
genre = Comedy  
AND  
year >= 2020
```

This is:

Metadata Filtering

Different systems may also call this scalar filtering or payload filtering.

A practical vector record therefore looks more like:

```
ID
+
VECTOR
+
METADATA
```

rather than just:

```
[0.21, 0.54, ...]
```

52. Why Metadata Filtering Matters

Suppose semantic search returns:

Interstellar	Sci-Fi	2014
The Martian	Sci-Fi	2015
Gravity	Sci-Fi	2013
Apollo 13	Drama	1995

But the user requires:

```
genre = Sci-Fi
year >= 2014
```

The eligible results become:

```
Interstellar
The Martian
```

Exactly how metadata filtering interacts with ANN differs between systems.

Combining:

```
Vector similarity
+
Metadata filtering
```

efficiently is itself an important vector database engineering problem.

53. Dedicated Vector Databases

Examples of vector databases include:

```
Pinecone
Qdrant
Weaviate
```

These systems are designed around storing, indexing, and searching vectors efficiently.

54. pgvector

A traditional relational database can also gain vector capabilities.

With `pgvector`, PostgreSQL can store vectors using a vector column.

Conceptually:

```
CREATE TABLE movies (
  id BIGSERIAL PRIMARY KEY,
  title TEXT,
  embedding VECTOR(1536)
);
```

Nearest-neighbor search can then look conceptually like:

```
SELECT *  
FROM movies  
ORDER BY embedding <=> :queryVector  
LIMIT 10;
```

Indexes such as:

HNSW

and:

IVFFlat

can be created to accelerate vector search.

This means that using vector search does not always require adopting a completely separate database product.

Complete Mental Model

A practical vector search system can be understood as:

```
Original Data  
  ↓  
Embedding Model  
  ↓  
Vector  
  ↓  
Vector Storage  
  ↓  
Distance / Similarity Function  
  ↓  
ANN Index  
  ↓  
Candidate Retrieval  
  ↓  
Metadata Filtering
```

↓
Top K Results

At scale, compression may also be introduced:

Original Vector
↓
Quantization / PQ
↓
Smaller representation

Exact KNN vs ANN

Exact KNN

Query
↓
Inspect every vector
↓
Calculate similarity
↓
Return true Top K

Advantages:

- Mathematically exact
- Maximum possible recall
- Simple conceptually

Disadvantages:

- Expensive at large scale
- Query cost grows with dataset size

ANN

Query
↓

Use index
↓
Identify promising candidates
↓
Calculate similarities for those candidates
↓
Return approximate Top K

Advantages:

- Much faster at scale
- Examines far fewer vectors
- Practical for large real-world systems

Trade-off:

Speed ↔ Recall

IVF Mental Model

Vector Space
↓
Divide into regions
↓
Create centroid for each region
↓
Query arrives
↓
Find nearest centroids
↓
Search selected lists
↓
Return nearest candidates

Think:

Divide the world, then search the most promising neighborhoods.

HNSW Mental Model

```

Vectors
↓
Create graph connections
↓
Create hierarchical layers
↓
Query arrives
↓
Start at high layer
↓
Make large jumps
↓
Descend through layers
↓
Perform local search
↓
Return nearest candidates

```

Think:

Navigate the vector space instead of scanning it.

Product Quantization Mental Model

```

Large vector
↓
Split into smaller sub-vectors
↓
Cluster similar sub-vectors
↓
Create codebooks
↓
Replace each sub-vector with centroid ID
↓
Store small integer codes

```

Think:

Instead of storing every coordinate exactly, store which representative pattern it most closely resembles.

Final Picture

The major problems of large-scale vector search are:

1. SEARCH COST

Solved using ideas such as:

ANN
IVF
HNSW

2. STORAGE COST

Reduced using techniques such as:

Quantization
Product Quantization

3. REAL APPLICATION CONSTRAINTS

Handled using:

Metadata
Filtering
IDs
Application data

Together, these pieces form the foundation of modern vector database systems.

The key idea is not that vector databases magically understand meaning.

The embedding model creates the vector geometry.

The vector database is responsible for storing that geometry and searching it efficiently.

Embedding Model

↓

Creates meaningful vector space

Vector Database

↓

Stores + indexes + searches that space efficiently